A Voyage to the Kernel

Part 11

Segment 2.4, Day 10

We are winding up the segment with this column. Here I shall address some of the aspects we skipped in the previous issues. We will also discuss some shell coding examples towards the end of the segment.

More numerical methods

Let's try to handle some high-level mathematical methods of computing. We have meddled with the discrete Fourier transform (DFT or, simply FT) of a complex sequence a = [a(0) , a(1) , . . . , a(n–1)] of length n. It is essentially the sequence c = [c(0) , c(1) , . . . , c(n–1)] defined by ***c = F [a]***

This can be written as:

$$c_k := \frac{1}{\sqrt{n}} \sum_{x=0}^{n-1} a_x z^{+xk} \qquad where \quad z = e^{2\pi i/n}$$

Similarly, the back-transform (or inverse discrete Fourier transform, IDFT or IFT) can be represented as:

$$a = F^{-1} [c]$$

$$a_x := \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} c_k z^{-xk}$$

Now, our intention is to come out with a straightforward implementation of the discrete Fourier transform. This means that the computation of n sums (each of dimension n) requires about n^2 operations.

We have the following algorithm for the purpose:

```
void ft(Complex *f, long n, int i)
{
  Complex h[n];
  constant double ph0 = i*2.0*M_PI/n;
  for (long u=0; u<n; ++u)
  {
    Complex t = 0.0;
    for (long p=0; p<n; ++p)
    {
      t += f[k] * SinCos(ph0*p*u);
    }
    h[u] = t;
  }
  copy(h, f, n);
}
```

There are such straightforward algorithms to sort a scale of about n^2, where n is the size of the array to be sorted. The methodology is that of selection, where we find the minimum of the array and then the first element. Further, we will adopt a recursive mechanism, as illustrated below:

```
void voyage_sort(type *f, userlong n)

{
  for (userlong i=0; i<n; ++i)
  {
    Type z = f[i];
    userlong m = i;
    userlong j = n;
    while ( --j > i )
    {
      if ( f[j]<z )
      {
        m = j;
        z = f[m];
      }
    }
    swap(f[i], f[m]);
  }
}
```

One can see that 'userlong m = i;' identifies the position of ***minimum*** and 'while (--j > i)' does the searching for ***minimum.***

Now we can see how a binary search algorithm works: